

CODE TOUR · LOVE BUTTON 1.0 · SEPTEMBER 2026

Reading the Love Button

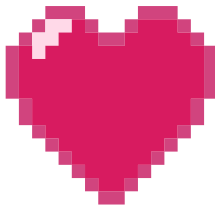
Twenty-four sittings through the whole codebase: which file to open, what to look for inside it, and why it is written that way. Plain words first, then the technical detail.

24 steps

2 languages

~6,000 lines

no prior Android needed



The tour

PART 0 • BEFORE YOU OPEN ANYTHING

1	The map: what is where, and why there are two halves	3
---	--	---

PART 1 • THE SERVER, FRONT TO BACK

2	The front door: every request starts here	4
3	What the code is handed at runtime	4
4	The two tables, in twenty lines of SQL	5
5	Who is calling: one hash, one row	6
6	Enrolling a phone	6
7	The send, line by line	7
8	Proving it to Google, and the push itself	8
9	The receipts, and why they cannot go backwards	8
10	Ceilings, and the nightly sweep	9
11	The server's tests, and how to run them	9

PART 2 • THE PHONE: WHAT A TAP DOES

12	How the app is wired together	11
13	What the phone remembers	11
14	The client: five calls, no framework	12
15	One tap, from finger to request	13
16	The ladder, and the races it survives	14
17	The twenty seconds	14

PART 3 • THE PHONE: WHAT ARRIVES

18	The push lands, and the sound plays	16
19	Seen: deciding whether anyone is looking	17
20	The shared bubble, and the clock both phones trust	17

PART 4 • WHAT YOU ACTUALLY SEE

21	The home screen, budgeted to the pixel	19
22	The pixel art, from ASCII to animation	20
23	Enrol, setup and the guide	21

PART 5 • PROVING IT WORKS

24	The app's tests, and one tap traced through every file	22
G	Glossary	25

How to use it. Each step is one sitting: open the files in the *Open* row, read the step, then read the file top to bottom. The steps are in dependency order — nothing asks you to already know something a later step explains — so going straight through is the fastest route, and a step you already know is safe to skip.

OPEN The files for this sitting. `send.ts:46` means line 46 of that file.

IN PLAIN WORDS What the file is for, with nothing technical in it.

READ IT IN THIS ORDER The numbered walk through the file. This is the step.

WHY The reason it is written that way rather than the obvious way.

GOT IT WHEN One question. If you can answer it, move on.

Underlined words are in the glossary at the back; on the web, tap one to read it in place. Paths are from the repository root. Code excerpts are real and trimmed; notes after + were added here.

Before you open anything

One sitting to learn the shape of the repository, so that every later step knows where it is.

Step 1 The map: what is where, and why there are two halves BOTH

OPEN `README.md` `love-button-spec.md` `server/wrangler.toml` `app/build.gradle.kts:1-40`

IN PLAIN WORDS

Two programs live in this repository. One runs on the two phones; one runs on Cloudflare's machines. They never share code — they share a small agreement about what a request looks like. Everything else in the repository is words about those two programs.

The tree, with the reason for each branch

```
app/           the Android app           Kotlin, ~3,000 lines in 38 files
src/main/java/com/lovebutton/app/
data/         storage and the network client
work/         background jobs
push/         receiving from Google
widget/       the four home-screen tiles
ui/           the screens you open
src/main/res/ pictures, sounds, fonts, widget metadata
src/test/     17 test files
server/       the Cloudflare Worker      TypeScript, 960 lines in 14 files
src/routes/   one file per endpoint
migrations/   the database schema, as .sql
test/         11 test files, 79 tests
docs/         the field manual, this tour, setup and secrets
scripts/      the pixel-icon generator, the PDF build, backups
```

Read it in this order

- 1 `README.md` — the front door. What the product is, in a screen's worth.
- 2 `love-button-spec.md` — the binding document, and the longest thing here. Read the architecture section and the numbered *invariants*; the rest is best read after the code rather than before it. An invariant is a promise the code keeps no matter what arrives: *identity comes only from the token, the recipient is derived, never sent, only the recipient may acknowledge*. You will meet all three in Part 1.
- 3 `server/wrangler.toml` — 26 lines that say what the server is. `main` names the entry file, `[vars]` holds public settings, the two `[[...]]` blocks attach the database and the cache, and `[triggers]` is the nightly cleanup. No password appears, which is the point of step 3.
- 4 `app/build.gradle.kts` lines 1–40 — how the app is built. `Gradle` is the app's build tool; this file reads the Worker's address out of `local.properties` (untracked, so a public repository does not carry it) and fails the build loudly when it is missing. Further down, `versionCode` and the release signing block — the comments there are worth the minute.

WHY IT IS BUILT THIS WAY

- A repository is a set of decisions. The spec records what was *not* built and why, which is the part a reader cannot reconstruct from the code. The code records the rest.
- Nothing secret is in here. The Google key, the two enrolment codes, the signing keystore and the Worker's address all live outside git. `docs/SECRETS.md` says where each one is.

GOT IT WHEN You can say which half holds the private key that can push to any phone (the server), and which half holds the words "Miss you" (the app). If those were the other way round, the design would be broken — step 13 says why.

1 The Worker, front to back

Ten sittings, 960 lines, and no Android knowledge needed. Read this half first: it is smaller, it is one language, and it defines the agreement the app has to keep.

Step 2 The front door: every request starts here SERVER

OPEN `server/src/index.ts` `all 54 lines`

IN PLAIN WORDS

When a phone asks the server for something, this file decides which piece of code answers. It also holds the one job that runs on a timer rather than because somebody asked.

Read it in this order

- `:1-8` the imports. Each `routes/...` import is one endpoint, and there are only four of them. That is the whole API.
- `:10` `new Hono<App>()`. Hono is a small framework whose entire job is *routing*: matching a method and a path — `POST /v1/send` — to the function that handles it. `<App>` is a type parameter that tells TypeScript what this app is carrying; step 3 defines it.
- `:17` `/health`. The simplest handler in the codebase: `c` is the *context* (this request and its reply), and `c.json({ok:true})` answers with JSON. Curl it and the deployment is proved alive.
- `:19-22` four `app.route("/v1", ...)` lines, which mount each routes file under `/v1`. Versioning the path from day one costs nothing and leaves room for a `/v2` that never has to come.
- `:27-32` `notFound` and `onError`. Without them Hono answers a mistyped path with plain text, and the app's parser — which expects JSON, always — would fail on the reply rather than on the error. `onError` also logs the real exception and returns a message that says nothing about it.
- `:41-53` the `default export`. Not the Hono app itself but an object with two keys, because a Worker can only expose a `scheduled` handler from an object: `fetch` is "a request arrived", `scheduled` is "the clock said 03:17". Note the comment on `await`: the purge is awaited so a failure shows up as a failed run rather than vanishing.

GET IT WHEN You can name the six endpoints without scrolling: `/health`, and under `/v1`: `enroll`, `devices` (post and delete), `send`, `receipts`.

Step 3 What the code is handed at runtime SERVER

OPEN `server/src/env.ts` `server/src/http.ts` `server/wrangler.toml`

IN PLAIN WORDS

The server never opens a database connection or reads a password. Cloudflare hands it a database object and a cache object already opened, plus a handful of settings, and that bundle is called the environment.

Read it in this order

- `env.ts:9` `interface Env`. An *interface* is a description of a shape, checked while the code is compiled and gone by the time it runs. `DB` is the *D1* database, `TOKEN_CACHE` the *KV* cache: these are *bindings*, objects Cloudflare injects. There is no host, no port and no password anywhere in this repository because of them.
- `env.ts:13-21` the difference that matters: `[vars]` in `wrangler.toml` are public and committed — the Firebase project id, the two names. The three below are *secrets*, uploaded once with `wrangler secret put`, stored encrypted, never readable again.

- 3 `env.ts:25-37` `DeviceRow` is the shape of one row of the phones table, and `Vars` says the request carries one of those attached to it. `App` bundles both, and is the `<App>` you saw in step 2 — so `c.get("device")` is type-checked everywhere it appears.
- 4 `http.ts:12` `fail()`. Every error the API returns has one shape: `{ "error": "rate_limited", "message": "..."}` . The app has exactly one error parser as a result (step 14).
- 5 `http.ts:22` `nowSeconds()`. Every timestamp in this system is epoch seconds. JavaScript counts milliseconds, so the division lives here once instead of in five call sites. The app, which thinks in milliseconds, multiplies at the boundary — watch for `* 1000L` in Part 3.
- 6 `http.ts:30` `readJson()`. Returns `null` instead of throwing when the body is missing, malformed, or not declared as JSON. Callers turn `null` into a 400, so a broken body can never reach the database layer.

GOT IT WHEN You can explain why `ENROLL_CODE_1` appears in `env.ts` but not in `wrangler.toml`.

Step 4 The two tables, in twenty lines of SQL SERVER

OPEN `server/migrations/0001_init.sql`

IN PLAIN WORDS

Everything the server stores is two tables: one row per phone, one row per message sent. There is no table of users and no table of conversations, because there are exactly two people and no history.

```
CREATE TABLE devices (
  id          TEXT PRIMARY KEY,           ← a UUID
  person      INTEGER NOT NULL CHECK (person IN (1,2)), ← a third person is impossible
  auth_hash   TEXT NOT NULL UNIQUE,       ← SHA-256 of the bearer token
  fcm_token    TEXT,                     ← push address; NULL = send-only
  label       TEXT,
  created_at   INTEGER NOT NULL,         ← epoch seconds
  updated_at   INTEGER NOT NULL
);
```

Read it in this order

- 1 The header comment. It names the two tables that do *not* exist, which is the more interesting half.
- 2 `CHECK (person IN (1,2))`. A CHECK constraint is a rule the database enforces on every write. “Only two people” is therefore not a convention anyone has to remember — a third is rejected by SQLite itself. Step 7 leans on this: it does arithmetic on `person` and needs it to be 1 or 2.
- 3 `auth_hash ... UNIQUE`. The token itself is never stored — only its SHA-256. A stolen copy of this database yields nothing you can log in with.
- 4 `fcm_token TEXT`, with no `NOT NULL`. That nullable column is a whole feature: a row with no push address is a *send-only* device (step 6), and it disappears from every fan-out query by itself, because `WHERE fcm_token IS NOT NULL` does not match it.
- 5 The `sends` table. `id` is the `send_id` the phone minted — the phone chooses the primary key, which step 15 explains and step 7 validates. `delivered_at` and `seen_at` start empty and are filled by receipts.
- 6 The two `CREATE INDEX` lines. An index is a lookup shortcut; `idx_sends_from_time` exists for exactly one query, the hourly count in step 10.

WHY IT IS BUILT THIS WAY

- **Numbered migration files.** The schema is a list of `.sql` files applied in order, so a fresh database and a live one reach the same state — and the tests apply these same files to a throwaway database (step 11).

GOT IT WHEN You can say what happens if the app sends `person: 3` . (Trick question: it cannot. No request carries a person at all — step 7.)

Step 5 Who is calling: one hash, one row SERVER

OPEN `server/src/auth.ts` `server/src/crypto.ts`

IN PLAIN WORDS

Every request except two carries a long random string that only one phone has. The server hashes it, finds the matching phone, and from then on *that* is who is calling. Nothing in the request body can change the answer.

Read it in this order

- `auth.ts:15` `requireDevice` is `middleware`: a function that runs before the handler and can either stop the request or hand it on with `await next()` . Mounted per-route with `.use()` , which you saw at the top of each routes file.
- `auth.ts:16-22` the header: `Authorization: Bearer <token>` is the standard shape; anything else is a 401 before any work happens.
- `auth.ts:26-32` `hash`, then look up. Note `.bind(authHash)` : the value goes to the database *separately* from the query text, in place of the `?` . That is a prepared statement, and it is why SQL injection is impossible here — the value is never parsed as SQL. Every query in this codebase is written this way.
- `auth.ts:46` `c.set("device", device)` . From here on, `c.get("device")` is the only source of identity. Grep the routes for `from_person` and confirm it is never read from a body.
- `crypto.ts:17` `randomToken()` . 32 random bytes from `crypto.getRandomValues` , hex-encoded to 64 characters. The comment earns its place: `Math.random()` is predictable and must never mint a credential.
- `crypto.ts:44` `secureEquals()` . Comparing two strings with `===` stops at the first differing byte, so the time it takes leaks how much of a guess was right — a `timing attack` . Hashing both sides first gives two 32-byte values, and the loop then always runs 32 times whatever the inputs. Used on the enrolment codes in step 6.

GOT IT WHEN You can answer: the database leaks. What can the attacker do with `auth_hash` ? (Nothing: they would need the string that hashes to it.)

Step 6 Enrolling a phone SERVER

OPEN `server/src/routes/enroll.ts` `server/src/limits.ts:34`

IN PLAIN WORDS

Each phone is set up once. It presents a long code typed in by hand, and gets back a token of its own, which person it is, and the other person's name. The code says which of the two people you are — there are only two codes.

Read it in this order

- `:36-42` the rate limit. This is the only endpoint reachable without a token, so it is capped at five attempts an hour per IP address, counted in KV, with a `Retry-After` header saying when to come back.
- `:19` `personForCode()` . Both comparisons are started before either is awaited (`Promise.all`), so the reply takes the same time whichever code was presented — no early return to measure. Step 5's `secureEquals` does the comparing.
- `:62-66` the send-only case. A body with no `fcm_token` is allowed and produces a row with `NULL` there: a device that can send but can never receive. The comment explains what it is for — the overnight check needs a token of its own without a debuggable build shipping to a real phone.

- 4 :73–77 what is minted: a device id, a 256-bit token, its hash, and the clock.
- 5 :82–90 the write, as a **batch** — two statements sent together. The **DELETE** first: one phone has one push address, so an existing row with the same address *is* this phone re-enrolling, and leaving it would double every future push. The comment on `= NULL` is the kind of SQL detail that bites: in SQL, `NULL = NULL` is not true, so send-only rows never delete one another. That is the wanted behaviour, for free.
- 6 :93–98 the reply, carrying the raw token. This is the only moment it exists in readable form anywhere; the server keeps only the hash, and the phone stores it (step 13).

WHY IT IS BUILT THIS WAY

- **No pairing flow, no invitations, no accounts.** Two codes, set once by hand, each naming a person. The whole feature is 99 lines because the product refuses to have more than two users.

GOT IT WHEN You can say what a wrong code returns (403 `invalid_code`) and why the reply takes the same time as a right one.

Step 7 The send, line by line SERVER

OPEN `server/src/routes/send.ts` `server/src/messages.ts`

IN PLAIN WORDS

The one endpoint the whole product exists for. A phone says “message 3”; the server decides who that goes to, writes it down, and asks Google to wake the other phone. It never learns the words, and it is never told the recipient.

Read it in this order

- 1 :38 `recipientOf()` — the invariant in one line: `3 - person`. With two people the recipient is arithmetic, so there is no lookup to get wrong and no field to forge. A stolen token can only ever message its own partner.
- 2 :24 `isUuid()`. The phone mints the id, and the id becomes a primary key, so it is validated rather than trusted — a client that could write arbitrary keys could collide with rows it does not own.
- 3 :49–52 the hourly ceiling (step 10) before anything else, so a runaway loop is refused cheaply.
- 4 :54–61 validation: the body parses, `msg_id` is one of the four the server allows (`messages.ts` knows the numbers and deliberately not the words), `send_id` is a UUID if present.
- 5 :73–82 the write, and the cleverest four lines in the file:
- ```
INSERT OR IGNORE INTO sends (id, ...) VALUES (?, ?, ?, ?) ... if (inserted.meta.changes === 0) return fail(c, 409, "duplic
```
- Asking “does this id exist?” and then inserting would let two racing taps both pass the check. **INSERT OR IGNORE** lets the database decide, and `changes` — how many rows were actually written — is the only answer that cannot race. The row is written *before* any push, because a receipt arriving later has to find something to correlate against.
- 6 :84–88 the recipient’s devices: `WHERE person = ? AND fcm_token IS NOT NULL`. Her phones, minus any send-only rows.
- 7 :92–98 the payload. Five string fields, one of them `msg_id: "3"`. FCM data values are strings only, which is why `String(...)` appears. Note what is absent: the text.
- 8 :100–106 the fan-out. `Promise.all` starts every push at once and waits for all of them, rather than sending to her phones one after another — with a 10ms CPU budget, waiting in series would be the slow path for no gain. Priority `HIGH`, which is what wakes a phone in `Doze`.
- 9 :108–115 dead tokens deleted — but only the ones Google called `UNREGISTERED`, which step 8 explains in full.
- 10 :117–125 the reply: `{send_id, delivered, sent_at}`, always 200. A `delivered` of zero is not a failure, it is “she has no phone registered”, which the app words differently. `sent_at` is the server’s clock, and step 20 is entirely about why that one number matters.

**GOT IT WHEN** You can trace what two identical requests with the same `send_id` produce: one 200, one 409, and exactly one push.

## Step 8 Proving it to Google, and the push itself SERVER

OPEN `server/src/google-oauth.ts` `server/src/fcm.ts`

### □ IN PLAIN WORDS

Google will only push to a phone if the asker owns the project. The server proves that by signing a short note with a private key, swapping the note for a pass that lasts an hour, and keeping the pass in the cache until it expires.

### Read it in this order

- `google-oauth.ts:54 buildSignedJwt()` . A JWT is three base64url chunks joined by dots: a header saying how it is signed, a claims object, and a signature over the first two. The claims are worth reading literally — `iss` who is asking, `scope` what for, `aud` who it is for, `iat` / `exp` valid from and until.
- `:34 pemToArrayBuffer()` . The key in the service-account file is PEM text; `crypto.subtle.importKey` wants raw bytes. Strip the header and footer lines, un-base64 the middle.
- `:69-83` the signing itself, through `WebCrypto` — `RSASSA-PKCS1-v1_5` with SHA-256, which OAuth calls `RS256`. Workers have no Node `crypto` module; this is the standard browser API instead.
- `:105-128 mintAccessToken()` posts the signed note to `oauth2.googleapis.com/token` as a form, and Google returns a token good for an hour.
- `:138 getAccessToken()` — cache first, mint only when cold, store for 3,300 seconds (55 minutes, five of margin). Read the comment: this is *not* an optimisation. One RSA signature sits close to the free plan's entire 10ms CPU allowance per request, so signing on every send would break the deployment. Caching moves it to roughly one request an hour.
- `fcm.ts:64 sendPush()` . The body is the whole contract with Google: a token, a `data` map, and a priority. The comment above it is a warning — adding a `notification` block would hand the drawing to Android, bypassing the app's channels and with them the four sounds.
- `fcm.ts:40 isPermanentTokenFailure()` , and the long comment. Only the nested `details[].errorCode === "UNREGISTERED"` proves a token is dead. The outer status is derived from the HTTP code and can equally mean a mistyped project id — which fails for *every* phone at once, so trusting it would delete all her rows, including the one her bearer token is found by, and lock her out until she re-enrolled by hand.

**GOT IT WHEN** You can say what is in the cache, how long for, and what breaks without it.

## Step 9 The receipts, and why they cannot go backwards SERVER

OPEN `server/src/routes/receipts.ts`

### □ IN PLAIN WORDS

Her phone reports twice: “it arrived” and “she looked at it”. The server records each the first time only, and pushes the news back to the sender so his tile can light up.

### Read it in this order

- `:30-36` validation: a `send_id` , and a state that is exactly `delivered` or `seen` .
- `:45-47` unknown id → 404. The nightly sweep (step 10) removes rows after a week, so this is a normal answer, not a broken one.

3 :56-58 the third invariant: `row.to_person !== device.person` → 403. The sender *holds* the send id — he made it — so without this check he could forge a “seen” for his own message and light his own tile gold.

4 :68-82 the update, and the one word to take away:

```
UPDATE sends SET seen_at = COALESCE(seen_at, ?), delivered_at = COALESCE(delivered_at, ?) WHERE id = ?
```

`COALESCE` returns the first value that is not empty, so a timestamp already written wins and a repeated receipt changes nothing. That single word makes the endpoint *idempotent* and the state *monotonic* — and note that `seen` also fills `delivered_at`, because she cannot have read something that never arrived, even if the two reports raced.

5 :84-106 push the news to the sender at `NORMAL` priority. A receipt is not urgent, and normal priority costs far less battery on his phone.

**GOT IT WHEN** You can say what happens when `delivered` arrives *after* `seen` for the same message — on the server, and (step 16) on the sender’s tile.

## Step 10 Ceilings, and the nightly sweep

SERVER

OPEN `server/src/limits.ts` `server/src/retention.ts` `server/wrangler.toml:20-26`

### IN PLAIN WORDS

Two small files that keep the system from growing without limit: a cap on how often a person can send, and a job that deletes messages older than a week.

### Read it in this order

1 `limits.ts:8` `MAX_SENDS_PER_HOUR = 500`, and the comment calling it a circuit breaker rather than a usage limit. Human tapping never approaches it; a compromised phone or a loop does.

2 `limits.ts:14` the count: one `COUNT(*)` over the last 3,600 seconds — the query `idx_sends_from_time` exists for.

3 `limits.ts:34` the enrolment counter in KV, and the honest comment: KV is *eventually consistent*, so the count can lag under concurrent requests. That is acceptable because the codes are unguessable anyway; this is defence in depth, not the lock.

4 `retention.ts:15` `RETENTION_DAYS = 7`, with the reasoning: rows exist for receipt correlation (twenty seconds) and the hourly ceiling (one hour). Seven days is margin for debugging, not a history anyone relies on.

5 `retention.ts:29` `purgeOldSends(env, now = nowSeconds())`. The injectable clock is a testing seam: the test passes a fixed `now` and checks the boundary without waiting a week. Note `<` rather than `<=` — a row exactly on the cutoff survives one more night, and a test pins that so it stays a decision.

6 `wrangler.toml:26` `crons = ["17 3 * * *"]` — *cron* for 03:17 every day. The comment explains the odd minute: schedules on the hour queue behind everyone else’s on the platform.

**GOT IT WHEN** You can say why a skipped nightly run needs no catching up. (The query is a cutoff, not a cursor: the next run deletes slightly more.)

## Step 11 The server’s tests, and how to run them

SERVER

OPEN `server/vitest.config.ts` `server/test/apply-migrations.ts` `server/test/send.test.ts`

### IN PLAIN WORDS

Seventy-nine tests across eleven files, and they do not fake the server: they start the real Worker with a real throwaway database, send it real requests, and intercept only Google.

Read it in this order

- 1 `vitest.config.ts` — Vitest is the test runner, and `@cloudflare/vitest-pool-workers` makes it run each test *inside* workerd, the same runtime Cloudflare uses live. `readD1Migrations` loads the `.sql` files from step 4; the `bindings` block supplies stand-ins for the three secrets, so no real key is ever needed to run the suite.
- 2 `test/apply-migrations.ts` — three lines, run before each file: apply the migrations to a fresh in-memory D1. The schema under test is the schema that ships.
- 3 `test/send.test.ts:1-45` — the pattern. `SELF.fetch(...)` calls the Worker as a client would; `seed()` inserts a device row directly; `fetchMock` intercepts the FCM endpoint, and `disableNetConnect()` means a test that accidentally reaches the real internet fails instead of passing quietly.
- 4 Then skim the rest for what they pin: `auth` (8) the 401s, `enroll` (12) codes and rate limits, `send` (15) the ceiling, duplicates and fan-out, `receipts` (8) the 403 and the monotonic clamp, `fcm` (5) that only UNREGISTERED deletes a device, `retention` (7) the cutoff boundary, `crypto` (9), `messages` (6), `schema` (4) the CHECK constraint, `health` (2).

The commands

| COMMAND                             | DOES                                                                                                                                |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>npm test</code>               | Runs the suite once. In this WSL checkout <code>npx</code> is broken, so use <code>node node_modules/vitest/vitest.mjs run .</code> |
| <code>npm run typecheck</code>      | TypeScript with no output — catches what the tests do not.                                                                          |
| <code>npm run dev</code>            | The Worker on localhost, with a local database.                                                                                     |
| <code>npm run deploy</code>         | Ships it. Seconds, worldwide.                                                                                                       |
| <code>npm run migrate:remote</code> | Applies new <code>.sql</code> files to the live database.                                                                           |

**GOT IT WHEN** You have run the suite and watched 79 pass. That is the fastest proof you have the whole server working on your machine.



## What a tap actually does

Six sittings from the manifest to the request leaving the phone. Kotlin reads close enough to any C-family language that you can follow it cold; the three ideas worth knowing before you start are *suspend functions*, *Flow* and *WorkManager*, and each is explained where it first appears.

### Step 12 How the app is wired together

PHONE

OPEN `app/src/main/AndroidManifest.xml` `LoveButtonApp.kt` `MainActivity.kt`

#### IN PLAIN WORDS

An Android app is not a program with one beginning. It is a handful of parts the system can start independently — a screen, a push handler, four home-screen tiles, a reboot listener — and the manifest is the list of them. Any one of them can be the reason the app comes to life.

#### Read it in this order

- 1 `AndroidManifest.xml` top — four `uses-permission` lines, each with its reason. `INTERNET`, `RECEIVE_BOOT_COMPLETED` (redraw tiles after a reboot), `POST_NOTIFICATIONS` (Android 13 and later make this a question the user answers), and the battery-optimisation one whose comment explains why a sideloaded app may ask for it.
- 2 `<application android:name=".LoveButtonApp">` — and `allowBackup="false"`, which keeps the token and the stored state out of Google's cloud backups.
- 3 The component list. One `activity` (the screen), one `service` for pushes, five `broadcast receivers`: four widget receivers — separate registrations so the launcher offers four choices, each with its own label and its own `@xml/widget_...` metadata — and `BootReceiver`.
- 4 `LoveButtonApp.kt`, all 17 lines. The `Application` class runs first whenever the process starts, *whatever* started it. So a push at 3am creates the notification channels and registers the unlock listener just as surely as the user opening the app does. Both lines are safe to repeat, which is what makes that work.
- 5 `MainActivity.kt:33-55` — splash installed before `super.onCreate`, then edge-to-edge window handling, then `setContent { }`, which is where `Jetpack Compose` takes over: you describe the screen as functions, and the framework redraws whatever the changed data touches.
- 6 `MainActivity.kt:77-100` `Root()` — the entire navigation model. Not enrolled → `EnrollScreen`; otherwise `HomeScreen`, with setup and guide as two booleans. Read the comment on `null`: “storage has not answered yet” is a third state, and conflating it with “not enrolled” would flash the code prompt on every launch.

**GOT IT WHEN** You can list the five ways this app's process can be started, and say what runs first in every one of them.

### Step 13 What the phone remembers

PHONE

OPEN `data/Prefs.kt` `data/Messages.kt` `data/DeliverySetup.kt`

#### IN PLAIN WORDS

Five small files in the app's private storage, each with one job. The most important holds the token this phone signs its requests with. None of them is a database, and none is a message history.

#### Read it in this order

- 1 `Messages.kt:29` `MESSAGES` — the catalogue: id, text, notification channel, sound file. This list is the thing the server does not have. A push carries `msg_id: 3` and this table turns it into words, a picture and a sound, locally, which is why the words never pass through Google.

- 2 `Prefs.kt:11` `preferencesDataStore(name = "love_button")` . `DataStore` is Android's small key-value file — a handful of values, written atomically, stored where only this app can read them.
- 3 `Prefs.kt:33-48` `enrolment` is a `Flow`: a stream that emits the current value and then emits again every time the file changes. A screen that collects it redraws by itself when enrolment appears. Note the all-or-nothing map — three keys present or `null` .
- 4 `Prefs.kt:50` `suspend fun current()` . A `suspend function` can pause while it waits for the disk and resume later without blocking the thread it was on. Anything that reads storage in this app is a suspend function; that is why background jobs can call it freely.
- 5 `DeliverySetup.kt` — its own file rather than a key in `Prefs` , and the comment says why: `clearEnrolment()` calls `prefs.clear()` , and these settings belong to the phone, not to the account.

| FILE           | HOLDS                                             | READ IN STEP |
|----------------|---------------------------------------------------|--------------|
| love_button    | Token, which person this phone is, partner's name | 13           |
| current_send   | The one latest message, for the app screen        | 20           |
| pending_sends  | Which tile each recent send belongs to            | 17           |
| unseen_sends   | Arrivals waiting for the next unlock              | 19           |
| delivery_setup | Whether the Xiaomi steps were confirmed           | 23           |

**GOT IT WHEN** You can say why the four messages' words live here and not on the server — and what would have to change to add a fifth message (this file, and one line in `server/src/messages.ts` ).

**Step 14** **The client: five calls, no framework** **PHONE**

OPEN `data/ApiModels.kt` `data/LoveButtonApi.kt`

**IN PLAIN WORDS**

One class holds every conversation the phone has with the server. It is small enough to read in a sitting, which was the point — the alternative was a library that would have saved a few lines and hidden the rest.

**Read it in this order**

- 1 `ApiModels.kt` — one `data class` per request and reply. `@Serializable` lets the compiler generate the JSON conversion; `@SerializedName("msg_id")` maps the wire's `snake_case` to Kotlin's `camelCase` , so both sides keep their own conventions.
- 2 `ApiModels.kt:28` `SendRequest` — two fields, and read what is missing: no recipient. The comment names the reason, and step 7 is the server side of the same decision.
- 3 `ApiModels.kt:36` `sentAt: Long = 0L` — a default, so an app talking to an older Worker still parses the reply. Zero then means “no server clock”, and step 20 handles that case explicitly.
- 4 `ApiModels.kt:65` `sealed interface EnrolResult` — a `sealed type`: a closed set of outcomes ( `Ok` , `InvalidCode` , `RateLimited` , `Failed` ) that the compiler forces the screen to handle. Three of them exist because the user can do something different about each.
- 5 `LoveButtonApi.kt:37` `post()` — `OkHttp` request building, and a comment worth reading twice: do not also set `Content-Type` by hand, because `OkHttp` derives it from the body and overwrites yours.
- 6 `:46` `execute()` — `withContext(Dispatchers.IO)` moves the blocking network call onto a thread pool meant for waiting.
- 7 `:51` , `:83` , `:101` , `:124` the four calls, and notice that each fails differently on purpose. `enrol` returns a result the screen can word; `registerDevice` returns a boolean; `send` throws, because its caller is a background job whose retry is the right response; `receipt` returns false without throwing, because a rejected receipt should not be retried.

## WHY IT IS BUILT THIS WAY

- The token is never logged. It is the only credential the phone has, and `adb logcat` is readable by anyone holding the phone.

**GOT IT WHEN** You can say why `send` throws while `receipt` returns a boolean.

## Step 15 One tap, from finger to request PHONE

OPEN `widget/MessageWidget.kt` `widget/LoveWidget.kt` `work/Sending.kt` `work/SendWorker.kt`

### IN PLAIN WORDS

The tile fills half-way the moment you touch it, and then nothing else happens on the spot: the tap writes down what it wants, hands the job to Android's background queue, and returns. On a phone that kills apps for fun, that queue is the only thing that survives.

### Read it in this order

- 1 `MessageWidget.kt:39` `provideGlance` — Glance is Compose for home-screen widgets: you write the same kind of function, and it is translated into the restricted views a launcher can draw. `currentState(KEY_STATE)` reads this tile's own stored state.
- 2 `MessageWidget.kt:46-73` `Tile()` — a full-size image with a click action, no text and no background. The state is carried entirely by which drawable is shown, plus one tint for the failed state (step 22).
- 3 `MessageWidget.kt:32-35` the two keys: where a tile stores its state, and the parameter a tap carries. `LoveWidget.kt:12` and `OtherWidgets.kt` then are four one-line subclasses — four registrations, one implementation.
- 4 `LoveWidget.kt:25` `SendAction.onAction` — the tap handler. Two statements: paint `SENDING`, then `beginSend`. Nothing blocking may happen here; this runs on the launcher's clock, and a slow request would freeze the home screen.
- 5 `work/Sending.kt:27` `beginSend()` — the hinge of the whole app, and only four statements. It mints the `send_id` and the tap time *here*, records the send, and queues two jobs. Read the comment: the worker is held back until the phone has a connection, so anything the worker owned would not have happened yet — a tap with no signal would have produced nothing on screen at all. Both tap paths, widget and app button, come through this one function.
- 6 `SendWorker.kt:40-58` `doWork()` — a `WorkManager` job: persisted to disk, survives the process being killed, retried when its constraints are met again. It reads the id and tap time out of its input data rather than making new ones, so a retry carries the original tap.
- 7 `SendWorker.kt:66` the abandonment check — `if (windowClosed(tappedAt)) return Result.success()`. If twenty seconds passed while this was queued, the send is dropped rather than delivered: the sender has already been told it did not happen, and buzzing her now would be a lie. Step 17 owns that window.
- 8 `:71-73` `pending.remember(sendId, appWidgetId)` — written *before* the request leaves, because a receipt can arrive before the reply does.
- 9 `:77-99` the call itself, then `update(sendId, SENT)`, then `markSentAt(...)` with the server's clock (step 20), then `settle`. The `catch` arm paints `FAILED` and asks for a retry.
- 10 `:114-133` `settle()` — show a state for its hold time, then return the tile to idle. `SENT` is the exception: it waits out the whole window because a receipt may still be coming, and deliberately does *not* clear the tile afterwards. The comment explains the rule that decides it: two writers racing on one tile at the same millisecond is a coin toss, so there is one writer, and it is `TimeoutWorker`.

**GOT IT WHEN** You can say what the id and timestamp would break if they were made inside `SendWorker` instead of in `beginSend`. (Two things: a tap with no signal shows nothing, and every retry resets the clock.)

## Step 16 The ladder, and the races it survives PHONE

OPEN `widget/WidgetState.kt` `widget/WidgetStateWriter.kt`

### IN PLAIN WORDS

A tile has six states and climbs through them in one direction. Two files: one says what the states are and which way is up, the other writes them into a tile without two writers standing on each other.

### Read it in this order

- 1 `WidgetState.kt:11` the six: `IDLE`, `SENDING`, `SENT`, `FAILED`, `DELIVERED`, `SEEN`.
- 2 `:20` `holdMillis` — how long each is shown. `SENDING` has none (it ends when the request ends). `SENT` and `DELIVERED` hold for the whole twenty-second window, because something may still arrive; `SEEN` is terminal, so four seconds is enough; `FAILED` three.
- 3 `:44` `fromName()` — an unrecognised stored name becomes `IDLE` rather than throwing. A tile's store outlives reinstalls, so a name written by a different build can come back, and an exception inside a widget update leaves a blank tile that no tap can fix.
- 4 `:55` `isAwaitingOutcome` — only `SENDING` and `SENT` are waiting on something that might never come, so only those two can time out.
- 5 `:72-94` `ladderRank` and `advancesTo`, the heart of the file. The rank is written out explicitly rather than using the enum's own order, so that reordering the declaration for readability cannot silently change which receipt wins. And the comment records the bug that caused it: on hardware, `seen` arrived 54ms *before* the `delivered` for the same message, and written in arrival order the late `delivered` undid the gold.
- 6 `WidgetStateWriter.kt:19 - :33` the pair: `setWidgetState` writes unconditionally (a reset is not a step backwards), `advanceWidgetState` writes only if it advances. Receipts use the second.
- 7 `:40-60` the private writer. Two details to notice. It takes a platform `appWidgetId`, an `Int`, rather than Glance's own `GlanceId`, because an `Int` can be put into a job's input data and survive the process dying. And the compare-and-write both happen inside one `updateAppWidgetState` block, so no concurrent writer can land in between.
- 8 `:69-77` the fan-out. Each widget class owns its own instances, so the redraw is offered to all four and only the class that actually holds this id repaints. Asking the wrong one writes the state and quietly draws nothing.
- 9 `:103` `failWidgetStateIfWaiting` and `:126` `clearWidgetStateIf` — both conditional, both returning early when they changed nothing. The comments spell out the race each one closes: only the writer whose own value is still on screen is allowed to clear it.

**GOT IT WHEN** You can predict the tile for this sequence: `SENT`, then `seen`, then a late `delivered`. (Gold, and it stays gold.)

## Step 17 The twenty seconds PHONE

OPEN `data/PendingSends.kt` `work/TimeoutWorker.kt`

### IN PLAIN WORDS

A tap is worth following for twenty seconds. Inside that window a receipt lights the tile it came from; outside it, the tile gives up and goes grey, and late news is dropped. One constant, and one small file that remembers which tile each send belongs to.

### Read it in this order

- 1 `PendingSends.kt:16` `PENDING_WINDOW_MS = 20_000L`, imported by everything that needs it. Three places had their own copy once; the preflight review caught it before it could drift.

- 2    `:32 windowClosed(at, now)` — one definition of “too late”, and note the negative-age case: a clock correction going backwards must not grey out a send made a second ago.
- 
- 3    `:50 remember()` — entries are `"appWidgetId:timestamp"` keyed by send id. A separate file from the enrolment, deliberately: disposable correlation state should never share a file with the thing whose loss costs a re-enrolment.
- 
- 4    `:67 widgetFor()` — returns `null` for an expired entry. The comment warns of the trap: this can never be used to decide what to do *after* waiting out the window, because by then every entry is expired by construction.
- 
- 5    `:85 remainingMs()` — how long is left of the original window, used to hold a `delivered` tile lit for exactly as long as a `seen` could still match it.
- 
- 6    `:101 forgetExpired()` — without it the store grows forever, one entry per send that never got a receipt.
- 
- 7    `TimeoutWorker.kt:41-52` — twenty seconds after the tap, mark the tile failed if it is still waiting, hold three seconds, clear. Both writes are conditional, so a receipt that arrived first makes the whole job a no-op.
- 
- 8    `TimeoutWorker.kt:65 enqueue()` — and the most important line in the file is the one that *is not* there. This job carries no network constraint, on purpose: the send it shadows waits for a connection, so a timeout that waited for the same thing would never fire in exactly the case it exists for.

**GOT IT WHEN** You can say why the app screen needs no `TimeoutWorker`. (It derives the timeout every time it reads — step 20.)



## What arrives, and what it does

Three sittings on the other half of the round trip: the push landing, the decision that she has seen it, and the one record both phones agree on.

### Step 18 The push lands, and the sound plays PHONE

OPEN

push/PushService.kt

push/Notifications.kt

#### □ IN PLAIN WORDS

Google wakes the app and hands it five short strings. The app looks up which message that is, posts a notification on that message's own channel — which is what makes the right sound play — and tells the server it arrived.

#### Read it in this order

- 1 `PushService.kt:27` — the class extends `FirebaseMessagingService`, and the manifest entry from step 12 is what lets Play Services start it. Because the Worker never sends a `notification` block, every message lands in this code rather than being drawn by Android.
- 2 `:29 onNewToken` — push addresses rotate. The new one goes through a job (step 19), not an inline request.
- 3 `:35–44` the `"msg"` branch: parse `msg_id`, `from_name`, `send_id`, `sent_at`. Seconds on the wire, milliseconds in the app — there is the `* 1000L`.
- 4 `:45–61` order matters here: post the notification *first*, then report `delivered`. A receipt for a notification that failed to post would be a lie.
- 5 `Notifications.kt:49 ensureChannels()` — four channels, one per message, created at every start-up (creating an existing channel does nothing). Since Android 8 the channel owns the sound, not the notification, which is the entire reason four channels exist.
- 6 `Notifications.kt:35 soundUriString()` — and read its comment, which is the best small trap in the codebase. A channel freezes its sound at creation and can never be changed. Resource *ids* are renumbered whenever resources are added, so an id-based sound URI silently stops resolving one day — and Android's response to a sound it cannot resolve is silence, with no error anywhere. The name-based form survives renumbering.
- 7 `Notifications.kt:88 postMessageNotification()` — title is the sender's name, text is the message, icon is a flat heart silhouette tinted crimson. The comment explains why the tray gets a silhouette rather than the full artwork: Android builds the small icon from the drawable's transparency alone and throws the colours away.
- 8 `:130–134` a new notification id each time, so four taps stack as four messages instead of one that keeps changing.
- 9 Back to `PushService.kt:63–101`, the `"receipt"` branch: never a notification (a phone that buzzes when she reads a message is a phone nobody wants), update the app's record first, then look up the tile, advance it, and hold it lit for whatever the window has left.

GOT IT WHEN You can say what the push actually contains, and why changing a sound means deleting a channel.

## Step 19 Seen: deciding whether anyone is looking PHONE

OPEN

push/Presence.kt

push/UnlockReceiver.kt

data/UnseenSends.kt

work/ReceiptWorker.kt

### IN PLAIN WORDS

“Seen” does not mean she tapped the notification — a message read on the lock screen and swiped away was still read. It means the phone was awake and unlocked. If it was not, the message waits, and the next unlock reports it.

### Read it in this order

- Presence.kt:15** `couldBeLookingNow(interactive, keyguardLocked)` — two booleans, both required. A lit screen still behind the lock screen means the phone woke *for* the notification; an unlocked phone with the screen off is one in a pocket. The pure two-argument version exists so it can be tested without a device; `:24` is the wrapper that asks the real phone, and returns false if either system service is missing rather than guessing.
- PushService.kt:108-123** `reportOrRememberSeen()` — report now, or write the id down. `runBlocking` here rather than a launched coroutine, and the comment is the reason: this runs on FCM’s thread, and the wakelock that keeps the phone awake ends when the method returns. A fire-and-forget write could lose the id in exactly the case it exists for — a sleeping phone.
- UnseenSends.kt:44** `drain()` — read and clear inside one edit, so two unlock broadcasts arriving together cannot both take the same ids.
- UnlockReceiver.kt:26-44** — on unlock, drain and enqueue a `seen` for each. `goAsync()` tells Android the receiver is still working, so the process is not reclaimed mid-drain.
- UnlockReceiver.kt:63-75** — registered in code, never in the manifest, and the comment says why: `ACTION_USER_PRESENT` is not on Android’s exemption list, so a manifest-declared receiver for it would look correct and never run. It is registered from the Application class, so the push that delivers the message is also what brings this listener up — which is the window that matters.
- ReceiptWorker.kt:27-45** — and read the retry policy carefully. A rejected receipt (403, 404) returns success, because retrying cannot help; only a thrown network error gets `Result.retry()`.
- RegisterTokenWorker.kt** — the same shape for a rotated push address.

**GO TIT WHEN** You can say what happens to a message that arrives while the phone is face-down on a table all night. (Notification and sound now, `delivered` now, `seen` at the next unlock.)

## Step 20 The shared bubble, and the clock both phones trust PHONE

OPEN

data/CurrentSend.kt

all 245 lines

### IN PLAIN WORDS

Both phones show the same thing in the big area at the top: the most recent message, whoever sent it. This file is how two phones with two different clocks agree on which message that is — the hardest small problem in the app.

### Read it in this order

- :24-36** `SendSnapshot` — six fields, and two of them are clocks. `at` is this phone’s own clock at the top, and feeds the timeout only. `serverAt` is the server’s clock, and feeds the ordering only. Keeping them apart is the design.
- :59-66** `receivedWins()` — four lines, deciding whether an arriving message takes the bubble. Server clock against server clock, never against the local one: two clocks pretending to be one would disagree exactly when it mattered. And because that clock is in *seconds*, two sends moments apart routinely carry the same value, so the send id breaks the tie. The tie-break is arbitrary, but it is a total order, and both phones hold both ids, so both break it the same way.

- 3    **:89-90** `displayState()` — the timeout, derived on every read rather than written by a timer. Two things fall out for free: the twenty-second mark needs no surviving process, and a send from an hour ago that nothing came back for reads grey the moment the screen opens.
- 
- 4    **:135** `start()` — a new send replaces the record outright. A new subject, not an update: it must not inherit the previous one's provenance or its place in the ordering.
- 
- 5    **:162** `update()` — two guards in one edit block: it must be the same send (a receipt can arrive after you have sent something else), and it must advance (step 16).
- 
- 6    **:188** `markSentAt()` — the reclaim, and the comment is worth reading twice. Until the server's reply lands, your own send has no server timestamp, so anything arriving takes the bubble unconditionally — even a message the server stamped *earlier*. Her phone, whose send did have a timestamp, keeps the newer one, and the two bubbles disagree. This takes it back, but only from something genuinely older and only if it is hers. It was written after the disagreement was seen on real phones.
- 
- 7    **:224** `receive()` — a message from her, stored as `SEEN`. That one choice buys the gold artwork, immunity from the timeout ( `SEEN` is not awaiting anything) and the top of the ladder, so no late receipt can move it. A seventh state would have had to be taught to the widget art, the guide and the ladder for something no widget can ever show.

**GOT IT WHEN** You can answer: both people tap within the same second. Which message ends up in both bubbles? (The one whose send id sorts higher — and crucially, the *same* one on both phones.)

## 4 PART 4 • THE FACE OF IT

# What you actually see

Three sittings on the screens and the artwork. Nothing here changes what the app does — and all of it is where the app stops feeling like a form with four buttons.

### Step 21 The home screen, budgeted to the pixel PHONE

OPEN `ui/HomeScreen.kt` `ui/Theme.kt` `ui/Sticker.kt` `ui/StateCopy.kt`

#### □ IN PLAIN WORDS

One screen: a big card at the top showing the latest message, four buttons with pandas standing on them, and two small buttons at the bottom. It never scrolls, which means everything has to fit — and that turns out to be the hardest part.

#### Read it in this order

- 1 `Theme.kt:18–36` — the Sticker Book: six colours, a hard ink keyline, and a 2dp offset shadow. Note what is *not* here: the ladder's state colours, which live in `PixelPalette` so the guide can never describe a colour the widget no longer draws.
- 2 `Sticker.kt:20` `StickerBox` — the shadow is a second box drawn behind the first, because Compose's own elevation shadow is soft and would not read as a sticker at all.
- 3 `StateCopy.kt:13–47` — one vocabulary for the six states, words and face kept separate so the focal area can animate the face while the guide prints the whole line. There were two vocabularies once; the guide then read as an explanation of a different screen.
- 4 `HomeScreen.kt:78–128` — the fixed heights, each with the reason it is fixed. The focal card does not wrap its content, because a card that grows by one line pushes the footer off the bottom of the screen.
- 5 `HomeScreen.kt:132` `fitHome()` — the screen is measured first, then three things give in order of what costs least: the air between stickers, then the focal card, then the pandas, each with a floor it cannot go below. The comment records the bug: a `Column` measures children in order, so the last child gets what is left — which was nothing, and the footer drew as two empty squashed stickers.
- 6 `:172–179` `LaunchedEffect` — the stored state does not change when a send times out, because the timeout is derived (step 20). So while the screen is open, nothing would prompt a redraw at the twenty-second mark: this sleeps exactly that long and then nudges one recomposition.
- 7 `:185–200` — the delivery-setup nudge, re-read on every `ON_RESUME` because the two Android checks are read live from the system and the user may have just changed them in Settings.
- 8 `:215–287` the focal area: the animated icon, the swaying state line, and a settled line that appears only on `SEEN` — in a slot that is always reserved, so its arrival moves nothing above it.
- 9 `:289–345` the four message stickers. The panda is a sibling drawn above the button, not inside it, because a sticker clips its own content and the panda's head has to break out over the top edge. It is deliberately not clickable, so taps fall through to the button underneath.
- 10 `:322–331` the tap itself: haptic feedback first, then `beginSend` — the same function the widget calls (step 15).

**GOT IT WHEN** You can say what changes on this screen when a receipt arrives, and what does not move.

## Step 22 The pixel art, from ASCII to animation PHONE

OPEN

scripts/pixel\_icons.py:1-40

widget/PixelGrids.kt

widget/WidgetArt.kt

ui/PixelArt.kt

ui/PixelLadder.kt

### IN PLAIN WORDS

Every icon in the app is a little picture typed out as text. One script turns those grids into the flat pictures the widgets show; the app redraws the same grids square by square when it wants to animate them. Both read the same source, so they cannot drift apart.

```
".....", ← '.' empty 'X' solid
"...XX...XX...", ← 's' shine (highlight)
"..XssXX..XXXX..", ← 'o' hole (inside the shape)
".XssXXXXXXXXXX.", the heart, from PixelGrids.kt
```

### Read it in this order

- 1 `pixel_icons.py` lines 1-40 — the format, and the rule that makes the whole thing hold together: the `outline` variant is derived (a solid cell touching an empty one), never drawn by hand. Run `python3 scripts/pixel_icons.py` and it writes 20 `VectorDrawables` into `res/drawable`, the notification silhouette, a preview sheet, and `PixelGrids.kt`.
- 2 `PixelGrids.kt` — generated, never edited by hand. Four grids, 16×16 and up.
- 3 `WidgetArt.kt:15 PixelPalette` — seven `ARGB` ints, plain numbers rather than Compose colours so that the widget, the Canvas and the guide can all read them without dragging in each other's dependencies.
- 4 `WidgetArt.kt:47 iconFor(msgId, state)` — the lookup from message and state to a drawable, shared by the widget and the guide screen. The guide explains what the widget draws, so it must be structurally unable to show something else.
- 5 `PixelArt.kt:31 isBorderCell()` — the same edge rule as the generator, and the comment records what happened when they disagreed on 22 cells: every eye and mouth got ringed in the dark border colour and the bubble's face read as uncanny.
- 6 `PixelArt.kt:125 cellColor()` — one cell, one state, one colour, and the whole ladder is visible in the `when`: idle draws only the outline, failed the same in grey, sent fills crimson, delivered pink, seen pink with a gold rim.
- 7 `PixelArt.kt:160 cellRect()` — rounding so neighbouring squares share an edge exactly. Without it, fractional positions leave hairline gaps and the white card shows through as pale lines between the pixels.
- 8 `PixelLadder.kt:32-68` the three animations, each a pure function that can be tested without a screen: `fillRowsVisible` (the fill rising bottom to top), `rippleReached` (delivered, expanding from the centre), `blinkAmount` (a blink every 3.4 seconds).
- 9 `PixelLadder.kt:141-184 borderRingOrder` and `gildOrder` — the gold light running round the outline on `seen`. To run *round* a shape you need the border cells in a walking order, so the code splits the border into touching pieces and walks each one, preferring the most constrained next cell. The paw's toes are separate blobs, which is why one ring was never enough.
- 10 `PixelLadder.kt:196 AnimatedPixelIcon` — and the first thing it does is ask whether animations are switched off system-wide, in which case every stage draws at rest.

**GOT IT WHEN** You can say why the app redraws the grid itself instead of showing the same `VectorDrawable` the widget uses. (A drawable cannot be lit one cell at a time, and lighting cells one at a time is the entire animation.)

## Step 23 Enrol, delivery setup, and the guide PHONE

OPEN

ui/EnrolScreen.kt

ui/SetupScreen.kt

device/DeviceSetup.kt

ui/GuideScreen.kt

### IN PLAIN WORDS

Three screens you see rarely: the one that sets the phone up, the checklist that makes sure messages can actually arrive on a Xiaomi, and the page explaining what the colours mean.

### Read it in this order

- 1 `EnrolScreen.kt` — a code field and a button. It asks Play Services for this phone's push address, calls `/v1/enroll`, and saves what comes back. The three outcomes from step 14's sealed type each get their own sentence.
- 2 `SetupScreen.kt:65` `deliveryReady()` — two checks Android will answer (notifications enabled, battery optimisation exempt) and, on a Xiaomi, one the phone will not: MIUI exposes no way to read Autostart, so it counts only once it has been confirmed by hand.
- 3 `DeviceSetup.kt:44` `startOrFallBack()` — MIUI's settings screens are private component names that differ between versions. Every one of them is opened through this, so a build that renamed the activity falls back to the app's own settings page instead of crashing.
- 4 `DeviceSetup.kt:106` `isNotificationPermissionPermanentlyDenied()` — a genuinely awkward API, explained in the comment: the flag it reads is false both *before* the first ask and *after* a permanent denial, so it is only meaningful immediately after a request came back denied.
- 5 `GuideScreen.kt:8-30` — the ladder in order, drawn with `iconFor(...)`: the widget's own pictures, unmodified. A guide that drew its own approximation would be teaching you about a picture that is not on your phone.

**GOT IT WHEN** You can say why the MIUI steps are a checkbox rather than a check.

## 5 Proving it works, and reading it whole

Step 24

The app's tests, and one tap traced through every file

BOTH

OPEN

app/src/test/~/SharedBubbleTest.kt

SendTimeoutTest.kt

WidgetStateTest.kt

### IN PLAIN WORDS

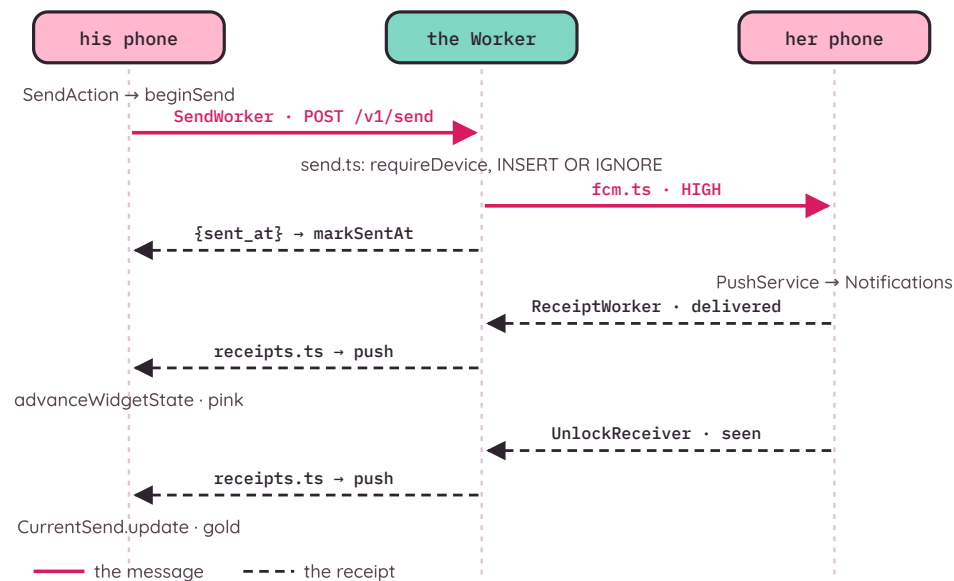
The app has 138 tests in 17 files. Almost all of them test the decisions rather than the drawing: which receipt wins, when a send is too old, what the bubble does when two messages arrive at once.

### How the app's tests are built

- 1 JUnit for the structure, `Robolectric` for anything that needs an Android class — it runs the framework on your laptop, so `DataStore`, `Context` and resources all work without a phone or an emulator.
- 2 `kotlinx-coroutines-test`'s `runTest` runs suspend functions and fast-forwards their delays, so a twenty-second window is tested in milliseconds.
- 3 `MockWebServer` stands in for the Worker in `LoveButtonApiTest`: a real HTTP server on localhost, so the client is exercised rather than mocked.
- 4 The pure functions — `advancesTo`, `windowClosed`, `receivedWins`, `cellColor`, `blinkAmount`, `swayOffsetPx` — exist as pure functions precisely so they can be tested like this. That is why so much logic sits outside the composables and the workers.
- 5 Run them with `./gradlew testDebugUnitTest`. They need a `JDK` and the Android SDK; the WSL checkout these documents were written in has neither, so the server suite is the one that runs here.

### Now trace one tap yourself

Open each file in turn and find the named function. This is the whole system in fourteen hops, and if you can follow it, you have read the app.



The same round trip as the field manual's, named by the file that does each part.

| #  | FILE                          | FIND                                                  |
|----|-------------------------------|-------------------------------------------------------|
| 1  | widget/LoveWidget.kt          | SendAction.onAction — paints SENDING, calls beginSend |
| 2  | work/Sending.kt               | beginSend — mints the id, records it, queues two jobs |
| 3  | data/CurrentSend.kt           | start — the app screen's record                       |
| 4  | work/SendWorker.kt            | doWork — window check, pending entry, the call        |
| 5  | data/LoveButtonApi.kt         | send — the only place the request is built            |
| 6  | server/src/auth.ts            | requireDevice — the token becomes a person            |
| 7  | server/src/routes/send.ts     | recipientOf , INSERT OR IGNORE , the fan-out          |
| 8  | server/src/google-oauth.ts    | getAccessToken — cache hit, almost always             |
| 9  | server/src/fcm.ts             | sendPush — data only, HIGH                            |
| 10 | push/PushService.kt           | onMessageReceived , the "msg" branch                  |
| 11 | push/Notifications.kt         | postMessageNotification — channel, sound, tray        |
| 12 | work/ReceiptWorker.kt         | doWork — delivered, then seen                         |
| 13 | server/src/routes/receipts.ts | the 403 guard, COALESCE , the push back               |
| 14 | widget/WidgetStateWriter.kt   | advanceWidgetState — the tile turns pink, then gold   |

## Then change something small

| TO CHANGE                 | TOUCH                                                                                                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Add a fifth message       | data/Messages.kt , a grid in pixel_icons.py , a sound in res/raw , a widget subclass and its manifest entry — and one line in server/src/messages.ts . |
| Change the twenty seconds | PENDING_WINDOW_MS , once. Everything else imports it.                                                                                                  |
| Change a sound            | The file in res/raw , and the channel has to be deleted and recreated — a channel's sound is frozen at creation.                                       |
| Change a state colour     | PixelPalette , then re-run pixel_icons.py so the drawables match the Canvas.                                                                           |

#### WHERE TO GO NEXT

`docs/manual/field-manual.html` explains what the app *does*, in the same voice as this. `docs/HOW-IT-WAS-BUILT.md` is how the work was done: 153 commits, eleven days, and what went wrong on real phones first. `love-button-spec.md` is the binding document behind both.

Every abbreviation and term this tour leans on. The field manual has its own for the parts of the system; this one is about the code.

### Activity

One Android screen, with a lifecycle the system drives. This app has exactly one, `MainActivity` .

---

**API** Application Programming Interface

The set of requests one program will answer for another. Here: six endpoints.

---

### Application class

Android object created before any other component in the process. Where one-per-process setup goes.

---

### ARGB

A colour packed into one 32-bit number: alpha, red, green, blue. `0xFFD81B60` is opaque crimson.

---

### base64url

Base64 with `+` and `/` swapped for `-` and `_` , so the result is safe inside a URL. What a JWT's three parts are encoded in.

---

### Binding

A resource Cloudflare hands the Worker as a ready-made object — `env.DB` , `env.TOKEN_CACHE` — instead of a connection string.

---

### Broadcast receiver

A component Android starts when something happens system-wide: a reboot, an unlock, a widget update.

---

### CHECK constraint

A rule the database enforces on every write. `CHECK (person IN (1,2))` makes a third person impossible.

---

### COALESCE

SQL for “the first of these that is not empty”. Used to keep the timestamp that was written first.

---

**Compose** Jetpack Compose

Android's way of writing screens as functions of data. Change the data and the framework redraws what depends on it.

---

### cron

A schedule written as five fields. `17 3 * * *` is 03:17 every day.

---

### D1

Cloudflare's SQLite database, reached through a binding.

---

### data class

A Kotlin class that is just its fields: equality, copying and printing come for free.

---

### DataStore

Android's small key-value file, written atomically and read as a Flow.

---

### Doze

Android's deep idle mode. It cuts idle apps off the network; only a high-priority push gets through.

---

### Epoch seconds

Seconds since 1 January 1970, UTC. One number, no time zone.

---

### Eventually consistent

A store where a write becomes visible everywhere shortly, not instantly. KV is one.

---

### export default

The one value a JavaScript module offers as its main thing. For a Worker, the object holding `fetch` and `scheduled` .

---

### Flow

A Kotlin stream of values over time. A screen collecting one redraws whenever it emits.

---

### Framework

A library that calls your code rather than the other way round. Hono, Compose and WorkManager all do this.

---

### Glance

Compose for home-screen widgets: the same style of code, translated into the limited views a launcher can draw.

---

## Gradle

The Android build tool. `build.gradle.kts` is a build script written in Kotlin.

---

## Hono

A small web framework for Workers. Matches a method and path to a function.

---

## HTTP method

The verb of a request: `GET` to read, `POST` to send something, `DELETE` to remove.

---

## Idempotent

Doing it twice has the same effect as doing it once. Both receipts are.

---

## Index

A lookup shortcut the database maintains so one shape of query stays fast.

---

## interface

In TypeScript, a description of a shape, checked at compile time and gone at runtime.

---

## IP address

The network address a request came from. Cloudflare passes it as `CF-Connecting-IP`.

---

## JDK Java Development Kit

What compiles and runs Kotlin and Java. Android Studio ships one.

---

## JSON

The text format for structured data used by every reply here: `{"ok":true}`.

---

## JWT JSON Web Token

Three base64url chunks joined by dots: header, claims, signature. Here it proves the Worker holds Google's private key.

---

## Kotlin

The language the app is written in. Runs on the Java virtual machine, with null-safety and coroutines built in.

---

## KV

Cloudflare's key-value store, used here as a cache with an expiry.

---

## Middleware

Code that runs before a handler and can stop the request or add to it. `requireDevice` is the only one here.

---

## Monotonic

Only ever moves one way. The ladder never goes back down.

---

## Notification channel

Since Android 8, the category a notification is posted on. The channel owns the sound, and it is frozen when the channel is created.

---

## OkHttp

The HTTP client the app uses. Small, and it handles connection reuse and headers.

---

## PEM

A key as text: base64 between `-----BEGIN-----` and `-----END-----` lines.

---

## Prepared statement

SQL with `?` placeholders, values sent separately. The value is never parsed as SQL.

---

## Primary key

The column that identifies a row uniquely. Here the phone chooses it, so the server validates it.

---

## Robolectric

Runs the Android framework on a laptop, so tests that need `Context` or resources need no phone.

---

## RS256

RSA signing with SHA-256. What Google accepts for a service-account JWT.

---

## sealed

A Kotlin type with a closed set of subtypes, so the compiler can check every case was handled.

---

## Secret

A value uploaded to Cloudflare encrypted, never committed and never readable again.

---

## Service

An Android component with no screen. `PushService` is started by Play Services when a push arrives.

---

**SHA-256**

A one-way fingerprint. Easy to compute, impossible to reverse, so it can stand in for a stored token.

---

**SQL injection**

An attack where text sent to a server gets run as SQL. Prepared statements make it impossible.

---

**SQLite**

A small, widely used database. D1 is Cloudflare's.

---

**suspend function**

A Kotlin function that can pause while it waits and resume later, without blocking its thread.

---

**Timing attack**

Guessing a secret by measuring how long each wrong guess takes to be rejected.

---

**Total order**

An ordering where any two items can be compared and everyone gets the same answer.

---

**Type parameter**

A type passed to another type, like `Hono<App>`, so the compiler knows what is inside.

---

**UNREGISTERED**

Google's answer for a push address that will never work again — the only one that deletes a device row.

---

**VectorDrawable**

An Android picture stored as shapes in XML. Scales to any size; cannot be animated cell by cell.

---

**Vitest**

The test runner the server uses. With Cloudflare's pool it runs tests inside the real Worker runtime.

---

**WebCrypto**

The standard cryptography API built into browsers and Workers. There is no Node `crypto` here.

---

**workerd**

The program that runs Cloudflare Workers, live and in the tests.

---

**WorkManager**

Android's queue for background jobs that must finish even if the app is killed. Jobs are saved to disk and retried.

---